Marketplace Blockchain Mondial 2030

Objectifs principaux et usages commerciaux

- Buts utilisateurs et fonction principale
  - Acheter des billets NFT avec propriété on-chain et transfert immuable assuré par le smart contract.
  - Revendre de manière sécurisée avec plafonnement et royalties pour éviter spéculation excessive.
  - Paiements en USDT pour stabilité financière et élimination de la volatilité crypto.
  - Vérification QR code unique pour authentifier billet et prévenir les contrefaçons.
- Exemples d'usage concret
  - Spectateur achète billet NFT, scanne QR pour accès le jour de l'événement; billet marqué comme consumé.
  - Revendeur met en vente, prix plafonné à 120% et royalties 5% reversées à l'organisateur.
  - Organisateur consulte historique on-chain pour vérification des ventes et répartitions.
- KPI et indicateurs clés
  - Taux de revente respectant plafond, nombre de billets scannés valides, volume USDT traité, incidents de doublons évités.

Architecture Technique détaillée

- Composants blockchain et smart contracts
  - Smart Contract 1: MondialTicketNFT.sol — ERC-721, génération NFT, QR unique, métadonnées équipe/catégorie/prix, anti-duplication.
  - Smart Contract 2: TicketMarketplace.sol — gestion listing, achat/vente, paiement USDT, royalties 5%, plafond revente 120%, historique transactions.
  - Tests et déploiement: usage Foundry pour tests unitaires, déploiement sur Sepolia/Polygon Amoy pour testnet.
- Backend Node.js et intégration blockchain
  - API Express.js avec routes GET /api/listings, GET /api/listing/:id, POST /api/buy, POST /api/list pour opérations marketplace.
  - Listener ethers.js surveillant événements TicketListed, TicketSold, ListingCanceled et synchronisant DB.
  - Connexion MongoDB pour indexation tickets, historique ventes et logs système.
- Stockage et base de données
  - MongoDB Collections: Tickets (tokenId, owner, price, qrCode, isListed), Historique des ventes, Logs système.
  - IPFS pour images billets et métadonnées JSON, liens stockés dans NFT metadata.

Frontend et expérience utilisateur

- Pages et navigation clés
  - Accueil (liste billets) affichant filtres et recherche.
  - Mes billets pour gestion personnelle, listing et transfert.
  - Achat/Vente pour processus d'achat et création d'annonces.
  - Historique pour consulter ventes et royalties.
- Intégration Web3 et paiement
  - Intégration MetaMask pour signatures et interactions on-chain.
  - Paiement USDT (ERC-20) via interaction avec TicketMarketplace.sol et approbation token.
  - Appels API REST pour listings, synchronisation avec backend et affichage état on-chain.
- Interfaces et sécurité UX
  - Vérification QR code côté frontend pour contrôle d'accès temps réel.
  - Indicateurs clairs pour prix plafonnés, frais et royalties.
  - Feedbacks transactions (pending/confirmed) et erreurs liées à gas/approbation.

Sécurité, avantages et problèmes résolus

- Mesures de sécurité et règles métiers
  - Royalties fixes 5% implémentées on-chain pour rémunération organisateur.
  - Plafond revente 120% pour limiter spéculation.
  - Paiement USDT pour stabilité; contrats vérifiés et backend indexé pour suivi.
- Avantages clés et bénéfices
  - Traçabilité complète via blockchain assurant transparence et auditabilité.
  - Authenticité garantie grâce aux NFT et QR codes uniques stockés on-chain.
  - Modèle économique durable avec royalties et contrôles anti-spéculation.
- Problèmes spécifiques adressés
  - Billets contrefaits remplacés par NFT authentifiés.
  - Spéculation excessive limitée par plafond 120%.
  - Absence de traçabilité résolue par immutabilité on-chain.
  - Volatilité limitée grâce au paiement stable en USDT.

Phases de développement et déploiement

- Phase 1: Smart Contracts (Mois 1-2)
  - Développement Solidity, implémentation ERC-721 et marketplace, tests unitaires avec forge test.
  - Déploiement sur testnet (Sepolia/Polygon Amoy) et vérification contrat.
- Phase 2: Backend & intégration (Mois 2-3)
  - Setup Node.js + MongoDB, création API REST, listener ethers.js, tests d'intégration et synchronisation events.
  - Mise en place logs et monitoring.
- Phase 3: Frontend & UX (Mois 3-4)
  - Développement React, intégration MetaMask, appels API, design UX/UI et tests utilisateurs.
  - Intégration visualisation historique et gestion QR.

Workflow Utilisateur et logique métier

- Parcours complet d'un billet
  - Organisateur crée billet NFT via smart contract; métadonnées et IPFS enregistrés.
  - Utilisateur A achète en USDT; propriété NFT transférée on-chain et DB mise à jour.
  - A peut lister pour revente; TicketMarketplace applique plafond 120% et royalties 5%.
- Distribution des fonds et historique
  - À la revente: A reçoit 95% du montant, organisateur 5% en royalties; transaction enregistrée on-chain et dans historique DB.
  - Historique visible via blockchain et page Historique frontend pour audit public.
- Validité et contrôle d'accès
  - QR code unique maintain validité même après revente; scan déduit ticket consommé et évite double entrée.
  - Backend indexé vérifie état isListed/isConsumed avant validation d'accès.